

Informe tecnico: replicacion de GoToGym Quantum Coach en GoToGym Developers

Fecha: 2026-06-09

Repos revisados: appdesplegada-inspect y gotogymdevelopers

Objetivo: determinar si el chat GoToGym Quantum Coach puede replicarse aqui y bajo que alcance.

Conclusion ejecutiva

Si, el Quantum Coach se puede replicar en gotogymdevelopers, pero no como copia directa. La app de referencia esta construida sobre Django REST + JavaScript vanilla, mientras que gotogymdevelopers usa Express/TypeScript + React/Vite. La estrategia correcta es portar el producto por capas: contrato de API, motor conversacional, memoria, experiencias QAF, interfaz React y observabilidad.

La opcion recomendada es una replicacion progresiva. Primero un MVP conversacional con /api/chat, Azure/OpenAI, contexto de coach ya disponible y quick actions basicas. Despues se migran experiencias QAF, memoria persistente, adjuntos, voz y analitica. Esto reduce riesgo y permite probar valor antes de traer todo el sistema.

DICTAMEN: Dictamen: viable con adaptacion media-alta. No es un "copiar y pegar"; es una migracion funcional hacia la arquitectura actual.

Que existe en appdesplegada

En appdesplegada el Quantum Coach es un sistema completo, no solo una ventana de chat. Esta compuesto por frontend, backend, motor QAF, memoria, runtime IA, experiencias especializadas, adjuntos, voz y trazabilidad.

- Frontend: frontend/js/chat.js inyecta el widget GoToGym Quantum Coach, maneja historial, drawer de memoria, quick actions, adjuntos, microfono, voz y flujos guiados.
- Endpoint principal: backend/api/urls.py registra path("/chat/", views.chat), equivalente a POST /api/chat/.
- Motor de decision: backend/api/chat_gateway.py decide CONTINUE, CONFIRM, START, EXPERIENCE_STEP o ATTACHMENT_ANALYSIS.
- Motor QAF: backend/api/qaf_cognition/* contiene decision layer, router cognitivo, emotional engine, memory engine, response policy, language governance y generadores.
- Experiencias QAF: calorías, meal planner, perfil metabolico, postura, lifestyle, motivacion, progresion, medicion muscular, skin health, shape presence y posture proportion.
- IA: backend/api/llm/azure_client.py integra Azure OpenAI Chat con deployment gpt-4.1-mini, API key o Managed Identity.
- Arquitectura: ARCHITECTURE.md documenta Azure OpenAI por Managed Identity, backend Django, PostgreSQL, Blob Storage, Front Door, Key Vault y Application Insights.
- n8n: aparece como legado/descomisionado; el runtime actual documentado es backend-first.

Que existe hoy en gotogymdevelopers

El repo actual ya tiene una base util, pero no contiene el Quantum Coach conversacional completo. Hay un frontend React/Vite, un backend Express/TypeScript y servicios de contexto/bienestar.

- Backend actual: backend/src/app.ts registra /api/integrations, /api/bodygraph, /api/v1/auth, /api/v1/smartwatch y /api/v1/business/wellbeing.
- No se encontro /api/chat ni un controlador conversacional equivalente al Quantum Coach.
- Si existe consumo de /api/coach_context/ desde frontend/src/services/wellbeingService.ts, pero es un cliente hacia el API de GoToGym, no el motor local del coach.
- docs/COACH_CONTEXT_ENDPOINT_MAP.md documenta un contrato fuerte de contexto de usuario: profile, documents, devices, if_snapshot, business y wellbeing_experience_value_v1.
- La consola frontend usa useCoachContext y componentes de cards para mostrar bienestar, lo cual sirve como insumo para personalizaci3n del chat.
- El package raiz ya incluye openai, pero backend/package.json no lo incluye. Si el chat vive en backend, conviene agregar el SDK alli o usar fetch contra el proveedor.

Brecha tecnica

Tabla: Area | appdesplegada | gotogymdevelopers | Brecha

Chat UI | Widget JS completo | React sin chat equivalente | Alta

Endpoint chat | POST /api/chat/ | No existe | Alta

IA | Azure OpenAI backend-first | openai en raiz, no integrado al backend | Media

Contexto usuario | /api/coach_context/ completo | Cliente y documentacion existentes | Baja-media

QAF experiencias | 13 experiencias y motores | Solo lectura/visualizaci3n de bienestar | Alta

Memoria | Conversational memory y share preview | No encontrada localmente | Alta

Adjuntos/OCR | Upload + OCR + Blob + vision | No encontrado | Alta

Voz | Preferencias, recomendaci3n y telemetría | No encontrado | Media-alta

Observabilidad | Headers X-Chat-* y scripts de pruebas | Basico | Media

Arquitectura recomendada para replicarlo aqui

La replicaci3n debe respetar la arquitectura de gotogymdevelopers. En lugar de traer Django, se recomienda crear un modulo Coach en Express/TypeScript y componentes React nativos.

- Backend: crear /api/v1/coach/chat con controller, service, router y tipos TypeScript.
- Contexto: reutilizar el contrato /api/coach_context/ como fuente inicial para perfil, IF, devices y wellbeing_experience_value_v1.
- IA: encapsular un CoachLLMService con proveedor configurable: OpenAI directo o Azure OpenAI. Variables sugeridas: OPENAI_API_KEY o AZURE_OPENAI_ENDPOINT, AZURE_OPENAI_DEPLOYMENT, AZURE_OPENAI_API_VERSION.
- Gateway: portar la logica esencial de chat_gateway.py a TypeScript: detecci3n de intenci3n explicita, guardrails de tercera persona/informacional, decision START/CONFIRM/CONTINUE y score.
- Frontend: construir un componente React QuantumCoachPanel con mensajes, input, quick actions, estado de carga, reset y consumo del endpoint local.
- Persistencia: iniciar con memoria de sesi3n/localStorage y luego mover a base de datos si el producto necesita continuidad real entre sesiones.
- Observabilidad: devolver metadata en JSON y headers equivalentes: gateway_decision, route, mode, runtime_source y fallback_reason.

Fases sugeridas

Fase 1 - MVP conversacional

- Crear /api/v1/coach/chat en Express.
- Agregar servicio LLM con prompts basados en build_quantum_coach_messages.
- Usar coach_context como contexto de entrada cuando el usuario este autenticado.
- Crear UI React con saludo, mensajes, quick actions y boton "Ver experiencias QAF".
- Alcance: conversacion y coaching general, sin activar experiencias complejas todavia.

Fase 2 - Gateway QAF y experiencias basicas

- Portar chat_gateway.py y compute_intervention_score a TypeScript.
- Implementar catalogo de experiencias y contratos de quick actions.
- Conectar 2 o 3 experiencias iniciales: lifestyle, progression y motivation, porque dependen menos de OCR o vision.

Fase 3 - Memoria y continuidad

- Guardar conversaciones por usuario/conversation_id.
- Implementar resumen de sesion y memoria selectiva.
- Agregar controles de privacidad: listar, borrar y no guardar.

Fase 4 - Adjuntos, vision y voz

- Agregar upload de imagen/PDF, validaciones, limites, OCR o extraccion de texto.
- Agregar speech-to-text/text-to-speech si el producto lo requiere.
- Replicar headers/trazas para QA y monitoreo.

Riesgos y cuidados

- Datos sensibles: coach_context puede contener salud, documentos, dispositivos, empresa y datos personales. La UI debe usar el bloque seguro wellbeing_experience_value_v1 siempre que sea suficiente.
- Costos IA: definir limites de tokens, timeouts, reintentos, fallback y telemetria antes de abrirlo a usuarios.
- No diagnosticar: el prompt debe mantener tono educativo y no sustituir atencion medica.
- Migracion parcial: copiar todo views.py de Django seria riesgoso. Lo sano es portar algoritmos pequenos y contratos.
- Compatibilidad: los nombres de endpoints deben alinearse con el prefijo actual /api/v1 para no romper la app.

Estimacion de esfuerzo

Tabla: Alcance | Tiempo estimado | Resultado

MVP chat + UI React | 3 a 5 dias | Coach conversacional funcional con contexto basico

Gateway QAF + 3 experiencias | 1 a 2 semanas | Activacion controlada y quick actions utiles

Memoria persistente | 1 semana | Continuidad entre sesiones y privacidad basica

Adjuntos + voz + vision | 2 a 4 semanas | Paridad alta con appdesplegada

Paridad casi completa | 4 a 8 semanas | Replica robusta con pruebas y observabilidad

Decision recomendada

Recomiendo replicar el Quantum Coach en gotogymdevelopers mediante un MVP backend-first en TypeScript, usando la identidad visual y UX de la captura, pero sin intentar migrar todo el legado de Django en un solo bloque. La prioridad debe ser demostrar una conversacion util, personalizada y segura; luego se incorporan QAF, memoria y adjuntos.

El primer entregable tecnico deberia ser: endpoint /api/v1/coach/chat, componente React QuantumCoachPanel, servicio LLM, gateway basico START/CONFIRM/CONTINUE, uso de coach_context y pruebas unitarias del router conversacional.

Referencias locales revisadas

- appdesplegada-inspect/ARCHITECTURE.md
- appdesplegada-inspect/STACK.md
- appdesplegada-inspect/docs/FLUJO_FUNCIONAMIENTO_IA_CHAT.md
- appdesplegada-inspect/docs/executive/acta_cierre_quantum_coach_v1.md
- appdesplegada-inspect/backend/api/urls.py
- appdesplegada-inspect/backend/api/chat_gateway.py
- appdesplegada-inspect/backend/api/llm/azure_client.py
- appdesplegada-inspect/frontend/js/chat.js
- backend/src/app.ts
- backend/src/services/corporate-wellbeing.service.ts
- frontend/src/services/wellbeingService.ts
- docs/COACH_CONTEXT_ENDPOINT_MAP.md